



**COMPARATIVE STUDY OF BAYESIAN AND  
NON-PARAMETRIC CLASSIFICATION**

SUBMITTED BY

Akorlie Edward Tsatsu (22424530)

PROJECT 1

DSCD612: Pattern Recognition (3 Credits)

MPhil/MSc Data Science

University of Ghana, Legon

September 2026

## Contents

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1. Problem Formulation and Dataset Description . . . . .           | 3  |
| 2. Exploratory Analysis and Preprocessing . . . . .                | 3  |
| 3. Feature Representation . . . . .                                | 7  |
| 4. Mathematical Explanation of the Principal Algorithms . . . . .  | 7  |
| 4.1 Sample mean and covariance . . . . .                           | 7  |
| 4.2 Bayes' rule and the Gaussian class-conditional model . . . . . | 8  |
| 4.3 k-Nearest Neighbours and distance measures . . . . .           | 9  |
| 5. Python Implementation . . . . .                                 | 9  |
| 6. Experimental Design . . . . .                                   | 11 |
| 7. Quantitative Evaluation and Comparison . . . . .                | 11 |
| 8. Visualization and Interpretation of Results . . . . .           | 13 |
| 9. Critical Discussion of Limitations . . . . .                    | 18 |
| 10. Central Research Question . . . . .                            | 19 |
| References . . . . .                                               | 20 |

**Live, executable notebook:** the full analysis below can be opened and re-run directly, no local setup required: Open in Google Colab. Source code and data: GitHub repository.

## 1. Problem Formulation and Dataset Description

This project addresses a **multiclass supervised classification** problem: given a vector of morphological measurements extracted from a photograph of a dry bean, predict which of seven cultivars the bean belongs to. Formally, we seek a function  $f: \mathbb{R}^{16} \rightarrow \{1, \dots, 7\}$  that maps a 16-dimensional feature vector  $x$  to a class label  $\omega_i$ , learned from a labelled training set and evaluated on held-out data.

The dataset used is the **UCI Dry Bean Dataset** (Koklu & Ozkan, 2020), obtained directly from the UCI Machine Learning Repository (<https://archive.ics.uci.edu/dataset/602/dry+bean+dataset>). It consists of high-resolution images of 13,611 dry bean grains belonging to seven registered Turkish bean cultivars, from which 16 geometric/morphological features (area, perimeter, axis lengths, shape factors, etc.) were extracted using a computer-vision pipeline. The full dataset shape used in this study is **13,611 rows  $\times$  16 features**, with the class label as a 17th column.

The seven classes and their observed frequencies in the full dataset are:

| Class    | Count | Share |
|----------|-------|-------|
| DERMASON | 3,546 | 26.1% |
| SIRA     | 2,636 | 19.4% |
| SEKER    | 2,027 | 14.9% |
| HOROZ    | 1,928 | 14.2% |
| CALI     | 1,630 | 12.0% |
| BARBUNYA | 1,322 | 9.7%  |
| BOMBAY   | 522   | 3.8%  |

This satisfies the assignment’s requirement of a dataset with at least three classes and a sufficient number of observations and features per class: seven classes are present, every class has well over a hundred observations even after a 70/30 train/test split, and 16 continuous features give both the Bayesian and k-NN classifiers enough information to separate the classes non-trivially while still allowing full covariance matrices to be estimated reliably (the smallest class, BOMBAY, still has 522 samples, comfortably above the  $16 + 1$  minimum needed to estimate an invertible  $16 \times 16$  covariance matrix).

## 2. Exploratory Analysis and Preprocessing

Two exploratory figures were produced directly from the raw feature matrix, before any scaling:

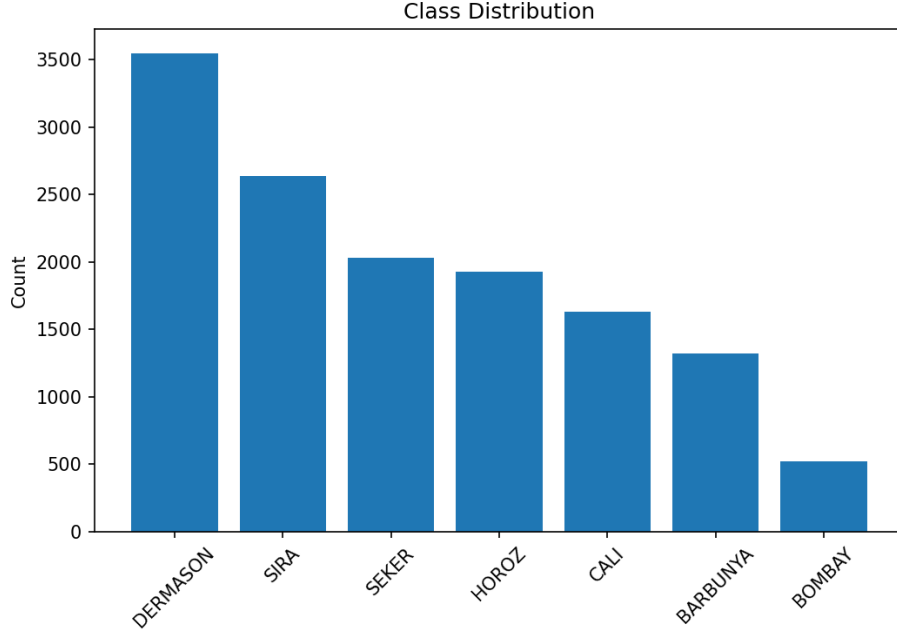


Figure 1: Class distribution

*Figure 1: Class distribution.* The bar chart confirms the class imbalance tabulated in Section 1: DERMASON is the majority class (3,546 samples) while BOMBAY is a small minority class (522 samples), a ratio of roughly 6.8:1. This imbalance is carried through the stratified train/test split (see below) and is revisited in Section 9 as a limitation.

*Figure 2: Feature correlation heatmap.* The  $16 \times 16$  Pearson correlation matrix computed over the full feature set shows extremely strong linear dependence among several groups of features, most notably:

| Feature pair                | Pearson r |
|-----------------------------|-----------|
| Area – ConvexArea           | 0.9999    |
| Compactness – ShapeFactor3  | 0.9987    |
| Perimeter – EquivDiameter   | 0.9914    |
| AspectRatio – Compactness   | −0.9877   |
| ConvexArea – EquivDiameter  | 0.9852    |
| EquivDiameter – Area        | 0.9850    |
| ShapeFactor3 – Eccentricity | −0.9811   |
| Perimeter – MajorAxisLength | 0.9773    |
| Compactness – Eccentricity  | −0.9703   |
| Area – Perimeter            | 0.9667    |

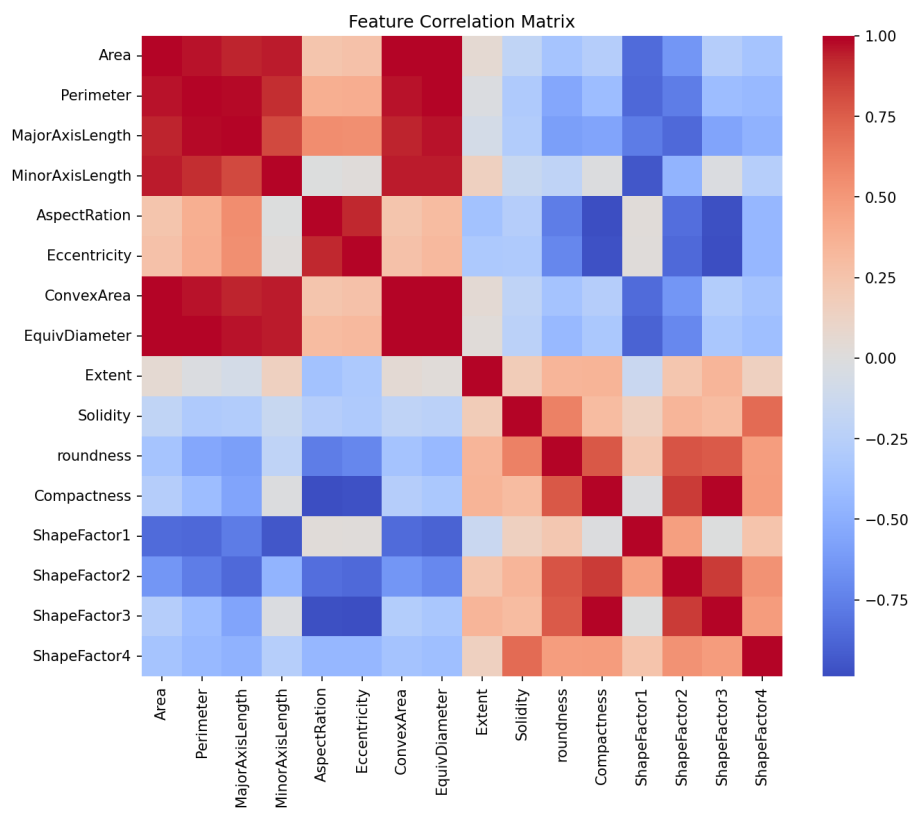


Figure 2: Correlation heatmap

Area, ConvexArea, EquivDiameter and Perimeter are all different mathematical transforms of the same underlying “how big is this bean” quantity, and move together (positive  $r$ ); Compactness, ShapeFactor3, AspectRatio and Eccentricity are all different transforms of “how elongated/round is this bean”, and here the relationships are inverse (negative  $r$ ): a more elongated bean (higher AspectRatio/Eccentricity) is by construction less compact and has a lower ShapeFactor3. Both groups still represent near-perfect *linear* dependence, just with opposite sign. This near-collinearity has a direct consequence for the Bayesian classifier: the per-class  $16 \times 16$  covariance matrix  $\Sigma_k$  is ill-conditioned in the directions spanned by these redundant feature groups; condition numbers computed directly on the fitted per-class covariance matrices range from roughly  $1 \times 10^7$  to  $1.3 \times 10^8$  across the seven classes, driven chiefly by the Area–ConvexArea collinearity ( $r = 0.9999$ , Figure 2). The implementation regularises each estimated covariance matrix by adding  $10^{-6}I$  before inversion (see `GaussianBayesClassifier.reg_covar` in `src/pattern_reg/bayesian_classifier.py`) as a defensive numerical-stability measure, not because the matrices are actually singular: refitting with `reg_covar=0` on this training split still yields a positive-definite covariance matrix for every class (`np.linalg.slogdet` sign = +1.0 throughout, just at those larger condition numbers), and test accuracy changes by only 0.02 percentage points (90.84% without the ridge vs 90.82% with it), i.e. the ridge measurably improves conditioning but is not load-bearing for invertibility or accuracy on this dataset.

**Global mean vector and covariance.** Over the full dataset the sample mean vector  $\mu \in \mathbb{R}^{16}$  and sample covariance matrix  $\Sigma \in \mathbb{R}^{16 \times 16}$  are computed by `pattern_reg.statistics.compute_mean` and `compute_covariance` (thin wrappers around `X.mean(axis=0)` and `np.cov(X, rowvar=False)`). The Bayesian classifier does not use these *global* statistics directly; instead `class_conditional_stats` computes one 16-dimensional mean vector and one  $16 \times 16$  covariance matrix **per class** (7 mean vectors, 7 covariance matrices in total), which is what a class-conditional Gaussian model requires (Section 4).

**Preprocessing.** The raw data contains no missing values after `dropna()` in `load_dry_bean_dataset` (13,611 rows survive unchanged). Preprocessing then proceeds as:

1. **Stratified 70/30 train/test split** (`sklearn.model_selection.train_test_split` with `stratify=y_encoded`, `random_state=42`), giving 9,527 training rows and 4,084 test rows while preserving each class’s proportion in both subsets (confirmed by the confusion-matrix row totals in Section 7: e.g. BOMBAY contributes exactly 157 of its 522 samples to the test set, i.e. 30.1%).
2. **Feature standardisation** with `sklearn.preprocessing.StandardScaler`, *fit on the training split only* and then applied to transform both the training and test splits. This is essential for k-NN, whose distance calculations would otherwise be dominated by features with naturally

large numeric ranges (Area is on the order of  $10^4$ – $10^5$  while Eccentricity is on  $[0, 1]$ ); fitting the scaler on the training data only, rather than on the full dataset, avoids leaking test-set statistics into the transform.

### 3. Feature Representation

Each dry bean grain is represented as a single 16-dimensional real-valued feature vector  $x = (x_1, \dots, x_{16})^\top \in \mathbb{R}^{16}$ , with components:

Area, Perimeter, MajorAxisLength, MinorAxisLength, AspectRatio, Eccentricity, ConvexArea, EquivDiameter, Extent, Solidity, roundness, Compactness, ShapeFactor1, ShapeFactor2, ShapeFactor3, ShapeFactor4.

These fall into three broad groups of derived geometric descriptors:

- **Size-based** measurements taken directly from the bean’s binary mask: Area, Perimeter, ConvexArea, EquivDiameter, MajorAxisLength, MinorAxisLength.
- **Shape-based** ratios that are (approximately) scale-invariant: AspectRatio (major/minor axis ratio), Eccentricity, Extent, Solidity, roundness, Compactness.
- **Shape factors** ShapeFactor1–ShapeFactor4, composite descriptors proposed in the original Dry Bean dataset paper that combine size and shape information into normalised, roughly scale-free indices.

Every feature is continuous, which is exactly the representation both classifiers under study require: the Bayesian classifier needs continuous inputs to fit a multivariate Gaussian density, and k-NN needs a metric space in which distances between feature vectors are meaningful. No categorical encoding or text processing is needed; the only representation-level transformation applied is the standardisation described in Section 2.

## 4. Mathematical Explanation of the Principal Algorithms

### 4.1 Sample mean and covariance

For a set of  $N$  feature vectors  $\{x_1, \dots, x_N\}$ ,  $x_i \in \mathbb{R}^d$  ( $d = 16$  here), the sample mean vector is

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$

and the (unbiased) sample covariance matrix is

$$\Sigma = \frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)(x_i - \mu)^\top$$

$\Sigma$  is a  $d \times d$  symmetric positive semi-definite matrix whose diagonal entries are the per-feature variances and whose off-diagonal entries are the pairwise covariances (Section 2's correlation table is simply  $\Sigma$  rescaled to  $[-1, 1]$  by feature standard deviations). Both are implemented in `pattern_reg.statistics.compute_mean / compute_covariance`, and per-class in `class_conditional_stats`, which produces  $\mu_k, \Sigma_k$  for each class  $k = 1, \dots, 7$ .

## 4.2 Bayes' rule and the Gaussian class-conditional model

Bayesian classification assigns a query point  $x$  to the class that maximises the posterior probability

$$p(\omega_i | x) = \frac{p(x | \omega_i) P(\omega_i)}{p(x)}$$

where  $P(\omega_i)$  is the class prior (estimated here as the empirical class frequency  $n_i/N$ ) and  $p(x | \omega_i)$  is the class-conditional likelihood. Since  $p(x) = \sum_j p(x | \omega_j) P(\omega_j)$  does not depend on  $i$ , the Bayes-optimal decision rule reduces to

$$\hat{\omega} = \arg \max_i p(x | \omega_i) P(\omega_i).$$

This project models each class-conditional density as a **multivariate Gaussian**:

$$p(x | \omega_i) = \frac{1}{(2\pi)^{d/2} |\Sigma_i|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_i)^\top \Sigma_i^{-1} (x - \mu_i)\right)$$

with a separate mean  $\mu_i$  and covariance  $\Sigma_i$  estimated per class (this is quadratic discriminant analysis / QDA rather than the equal-covariance special case, since each class is allowed its own  $\Sigma_i$ ). Substituting and taking logs for numerical stability gives the discriminant function actually implemented in `GaussianBayesClassifier.predict_log_proba`:

$$g_i(x) = -\frac{1}{2} \log |\Sigma_i| - \frac{1}{2} (x - \mu_i)^\top \Sigma_i^{-1} (x - \mu_i) - \frac{d}{2} \log(2\pi) + \log P(\omega_i)$$

with the predicted class  $\hat{\omega} = \arg \max_i g_i(x)$ . Because each class keeps its own covariance  $\Sigma_i$ , the term  $(x - \mu_i)^\top \Sigma_i^{-1} (x - \mu_i)$  (the squared **Mahalanobis distance** of  $x$  to class  $i$ 's mean) is quadratic in  $x$  and the resulting decision boundary between any two classes  $i, j$  (the set of  $x$  where  $g_i(x) = g_j(x)$ ) is, in general, a **quadratic hypersurface** in  $\mathbb{R}^{16}$ : an ellipsoid, paraboloid or hyperbolic surface depending on how  $\Sigma_i$  and  $\Sigma_j$  differ. Only in the special case  $\Sigma_i = \Sigma_j$  for all classes does the quadratic term cancel and the boundary reduce to a hyperplane (linear discriminant analysis).

### 4.3 k-Nearest Neighbours and distance measures

k-NN is a **non-parametric**, instance-based classifier: it makes no assumption about the functional form of  $p(x \mid \omega_i)$  and instead estimates the local class density directly from the training sample. Given a query  $x$ , k-NN computes the distance  $d(x, x_j)$  from  $x$  to every training point  $x_j$ , selects the  $k$  training points with smallest distance, and assigns  $x$  the majority label among those  $k$  neighbours (ties broken by label order in this implementation, `KNNCClassifier._majority_vote`).

Two distance measures were implemented and compared (`KNNCClassifier._compute_distances`):

**Euclidean ( $L_2$ ) distance:**

$$d_{\text{euclidean}}(x, x') = \left( \sum_{f=1}^d (x_f - x'_f)^2 \right)^{1/2}$$

**Manhattan ( $L_1$ , city-block) distance:**

$$d_{\text{manhattan}}(x, x') = \sum_{f=1}^d |x_f - x'_f|$$

The choice of distance measure changes *which* points count as “near” a query, and therefore changes the implicit shape of the decision boundary: Euclidean distance treats all directions in feature space symmetrically (its level sets are hyperspheres), which is appropriate when features are standardised and errors in different feature dimensions should be penalised quadratically. Manhattan distance sums absolute per-feature differences (its level sets are hyper-diamonds/cross-polytopes), which down-weights the influence of any single feature with an unusually large deviation relative to Euclidean distance, making it comparatively more robust to outliers in one or two dimensions but less sensitive to the *combined* effect of small deviations across many correlated dimensions, which is relevant here given the strong inter-feature correlation documented in Section 2. In addition to the distance metric, k-NN’s other key hyperparameter is  $k$  itself: small  $k$  (e.g.  $k = 1$ ) yields a highly flexible, low-bias/high-variance decision boundary that can overfit to noisy individual training points, while large  $k$  smooths the boundary (lower variance) at the cost of blurring genuine local class structure (higher bias). Section 6/7 sweep  $k \in \{1, 3, 5, 7, 9, 11, 15, 21\}$  under both metrics to characterise this trade-off empirically for this dataset.

## 5. Python Implementation

Both classifiers are implemented **from first principles in NumPy** (no `sklearn.svm`, `sklearn.naive_bayes`, `sklearn.neighbors` or other pre-built

classifier objects are used for the core algorithms), inside the installable `pattern_reg` package (`src/pattern_reg/`):

| Module                              | Responsibility                                                                                                                                                                                                                           |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>data_loading.py</code>        | Downloads/loads the Dry Bean dataset from the UCI repository into a pandas <code>DataFrame/Series</code> pair ( <code>load_dry_bean_dataset</code> ).                                                                                    |
| <code>statistics.py</code>          | Sample mean, covariance, correlation, and per-class conditional statistics ( <code>compute_mean</code> , <code>compute_covariance</code> , <code>compute_correlation</code> , <code>class_conditional_stats</code> ).                    |
| <code>preprocessing.py</code>       | Stratified train/test split and train-only <code>StandardScaler</code> fitting ( <code>prepare_experiment_data</code> ).                                                                                                                 |
| <code>bayesian_classifier.py</code> | <code>GaussianBayesClassifier</code> : from-scratch multivariate Gaussian Bayes/QDA classifier with log-space scoring and covariance regularisation.                                                                                     |
| <code>knn_classifier.py</code>      | <code>KNNClassifier</code> : from-scratch k-NN with pluggable Euclidean/Manhattan distance and majority-vote prediction.                                                                                                                 |
| <code>evaluation.py</code>          | Accuracy, macro precision/recall/F1, confusion matrix and classification report via <code>sklearn.metrics</code> (evaluation metrics only, not the classifiers).                                                                         |
| <code>visualization.py</code>       | Matplotlib figure generation: correlation heatmap, class distribution, PCA scatter, confusion matrices, accuracy-vs-k curve.                                                                                                             |
| <code>experiment.py</code>          | Orchestrates the full pipeline end-to-end: load data → preprocess → fit Bayesian classifier → sweep k-NN over <code>k × metric</code> → evaluate all configurations → persist <code>results/metrics/results.json</code> and all figures. |

`sklearn` is used only for well-established supporting utilities that are not themselves the classification algorithms under study (`train_test_split`, `StandardScaler`, `LabelEncoder`, and the metric functions in `sklearn.metrics`),

consistent with the exam’s requirement to implement the *classifiers* from first principles while using standard tooling for data handling and evaluation.

The package is fully unit tested: `pytest tests/ -v` reports **27 passed** tests covering the mean/covariance/correlation statistics against NumPy reference implementations, Bayesian classifier fitting and prediction (including a positive-definiteness/singleton-class failure path), k-NN prediction and metric-disagreement behaviour, evaluation metric correctness against hand-computed confusion matrices, preprocessing determinism and train-only scaling, the end-to-end experiment orchestration, and all visualization functions returning valid Matplotlib figures.

## 6. Experimental Design

The experiment (`pattern_reg.experiment`) follows a single, fixed protocol so that the Bayesian classifier and every k-NN configuration are compared on exactly the same data split:

1. Load the full  $13,611 \times 16$  Dry Bean dataset.
2. **Stratified 70/30 train/test split** (`random_state=42`): 9,527 training samples, 4,084 test samples, class proportions preserved in both subsets.
3. **Standardise** features using a `StandardScaler` fit on the training split only, applied to transform both splits (Section 2).
4. **Fit `GaussianBayesClassifier`** once on the standardised training data (per-class mean and regularised covariance), then evaluate once on the held-out test split.
5. **Sweep `KNNClassifier`** over the Cartesian product of  $k \in \{1, 3, 5, 7, 9, 11, 15, 21\}$  and metric  $\in \{\text{euclidean}, \text{manhattan}\}$  (16 total configurations), each fit on the same standardised training data and evaluated on the same held-out test split.
6. Every configuration (Bayesian + 16 k-NN variants) is scored with the same evaluation function (`evaluate_predictions`): accuracy, macro-averaged precision/recall/F1, and a full confusion matrix, all persisted verbatim to `results/metrics/results.json`.

Using one fixed train/test split and one fixed evaluation routine for every model removes split-related variance as a confound when comparing the Bayesian classifier against the k-NN sweep, so any performance differences observed in Section 7 can be attributed to the classifiers’ modelling assumptions rather than to different data being used.

## 7. Quantitative Evaluation and Comparison

All figures below are taken verbatim from `results/metrics/results.json` (test set size: 4,084 samples in every row).

| Classifier        | Config                    | Accuracy        | Precision<br>(macro) | Recall<br>(macro) | F1 (macro)      |
|-------------------|---------------------------|-----------------|----------------------|-------------------|-----------------|
| Bayesian<br>(QDA) | N/A                       | 0.908178        | 0.926770             | 0.923791          | 0.923873        |
| k-NN              | euclidean,<br>k=1         | 0.901322        | 0.918793             | 0.917257          | 0.918013        |
| k-NN              | euclidean,<br>k=3         | 0.916014        | 0.930948             | 0.928757          | 0.929818        |
| k-NN              | euclidean,<br>k=5         | 0.917483        | 0.932512             | 0.928608          | 0.930412        |
| k-NN              | euclidean,<br>k=7         | 0.916503        | 0.932451             | 0.927849          | 0.929973        |
| <b>k-NN</b>       | <b>euclidean,<br/>k=9</b> | <b>0.920176</b> | <b>0.935733</b>      | <b>0.931415</b>   | <b>0.933299</b> |
| k-NN              | euclidean,<br>k=11        | 0.918952        | 0.935565             | 0.930969          | 0.933012        |
| k-NN              | euclidean,<br>k=15        | 0.919442        | 0.936058             | 0.931867          | 0.933690        |
| k-NN              | euclidean,<br>k=21        | 0.919442        | 0.935911             | 0.931245          | 0.933289        |
| k-NN              | manhattan,<br>k=1         | 0.903771        | 0.919722             | 0.918465          | 0.919067        |
| k-NN              | manhattan,<br>k=3         | 0.915034        | 0.930969             | 0.928002          | 0.929389        |
| k-NN              | manhattan,<br>k=5         | 0.915769        | 0.932858             | 0.928581          | 0.930521        |
| k-NN              | manhattan,<br>k=7         | 0.915524        | 0.931995             | 0.927123          | 0.929304        |
| k-NN              | manhattan,<br>k=9         | 0.916014        | 0.932989             | 0.927567          | 0.929975        |
| k-NN              | manhattan,<br>k=11        | 0.917238        | 0.934492             | 0.928776          | 0.931364        |
| k-NN              | manhattan,<br>k=15        | 0.916748        | 0.933688             | 0.927592          | 0.930241        |
| k-NN              | manhattan,<br>k=21        | 0.916259        | 0.933308             | 0.927389          | 0.929965        |

**Best configuration overall: k-NN with Euclidean distance, k=9, accuracy = 0.920176** (3,758 of 4,084 test samples correctly classified), compared with the Bayesian classifier’s accuracy of 0.908178 (3,709 of 4,084 correct), a gap of **49 additional correct classifications**, or 1.2 percentage points, in k-NN’s favour. k-NN outperforms the Bayesian classifier on accuracy at 14 of the 16 tested configurations; the two exceptions are  $k = 1$  under both metrics (euclidean  $k=1 = 0.901322$ , manhattan  $k=1 = 0.903771$ ), both of which fall

below the Bayesian classifier’s 0.908178. The same 14 configurations also outperform the Bayesian classifier’s macro F1 (0.923873); again the two exceptions are euclidean  $k=1$  (F1 = 0.918013) and manhattan  $k=1$  (F1 = 0.919067). This is consistent with  $k = 1$  being the noisiest, highest-variance member of the  $k$ -NN sweep: with only a single neighbour vote the classifier is exposed to individual mislabelled or borderline training points in a way the Bayesian model’s smoothed class-conditional density is not, and it takes only a small increase to  $k = 3$  for  $k$ -NN to pull ahead of the Bayesian classifier on both metrics.

Within the  $k$ -NN sweep, accuracy rises steeply from  $k = 1$  to  $k \approx 5$ –9 and then plateaus (the four largest euclidean accuracies, at  $k \in \{9, 11, 15, 21\}$ , span a range of only 0.0012, from 0.918952 at  $k = 11$  to 0.920176 at  $k = 9$ ), indicating the classifier has reached a stable bias–variance trade-off well before  $k = 21$ . Euclidean distance outperforms Manhattan distance at every tested  $k$  except  $k = 1$ , where Manhattan is slightly better (0.9038 vs 0.9013; Manhattan is marginally more robust to the single noisiest neighbour). The size of the Euclidean advantage varies across  $k$ : it is smallest around  $k = 3/k = 7$  (about 0.1 percentage points) and largest exactly at the overall best configuration,  $k = 9$  (0.9202 vs 0.9160, a 0.42-point gap).

Examining the Bayesian confusion matrix (class order alphabetical: BARBUNYA, BOMBAY, CALI, DERMASON, HOROZ, SEKER, SIRA), the dominant error is 142 DERMASON test beans misclassified as SIRA (out of 1,064 DERMASON test samples), plus 38 SIRA beans misclassified as DERMASON and 33 BARBUNYA beans misclassified as CALI. BOMBAY is classified with **zero errors in every single configuration tested** (Bayesian and all 16  $k$ -NN variants); its row/column in every confusion matrix is a clean diagonal 157, because Bombay beans are geometrically an outlier class (visibly separated in the PCA plot, Section 8) despite being the smallest class by sample count.

## 8. Visualization and Interpretation of Results

*Figure 3: PCA projection of the standardised 16-dimensional feature space onto its first two principal components.* BOMBAY forms a visually distinct, well-separated cluster, consistent with its perfect classification accuracy across every model in Section 7: it is simply the largest bean by a wide margin and does not overlap the other six classes in feature space. In contrast, DERMASON and SIRA form adjacent, partially overlapping clusters in the PCA plot, which directly explains why 142 DERMASON beans are misclassified as SIRA under the Bayesian model (Section 7): their class-conditional Gaussians (or their local neighbourhoods, for  $k$ -NN) overlap substantially in this projection. This is also consistent with the strong size-feature correlations in Section 2 (Area/ConvexArea/Perimeter all  $\sim 0.97$ –1.0 correlated): DERMASON and SIRA differ mainly in these highly collinear “how big” features, giving the Bayesian model’s per-class covariance ellipsoids less independent information to separate them on than if the 16 features were less redundant.

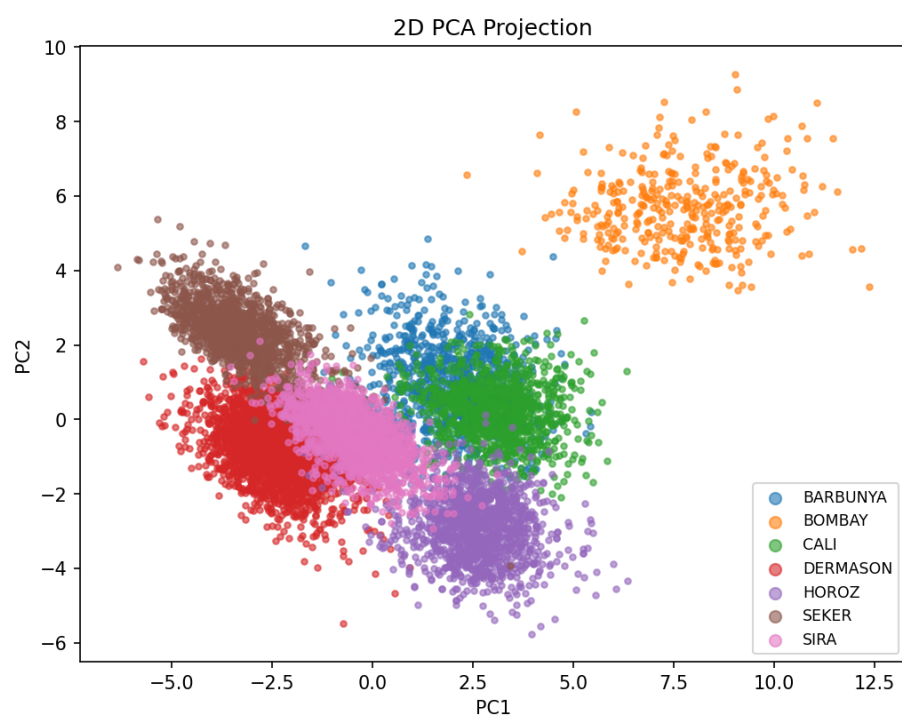


Figure 3: PCA scatter

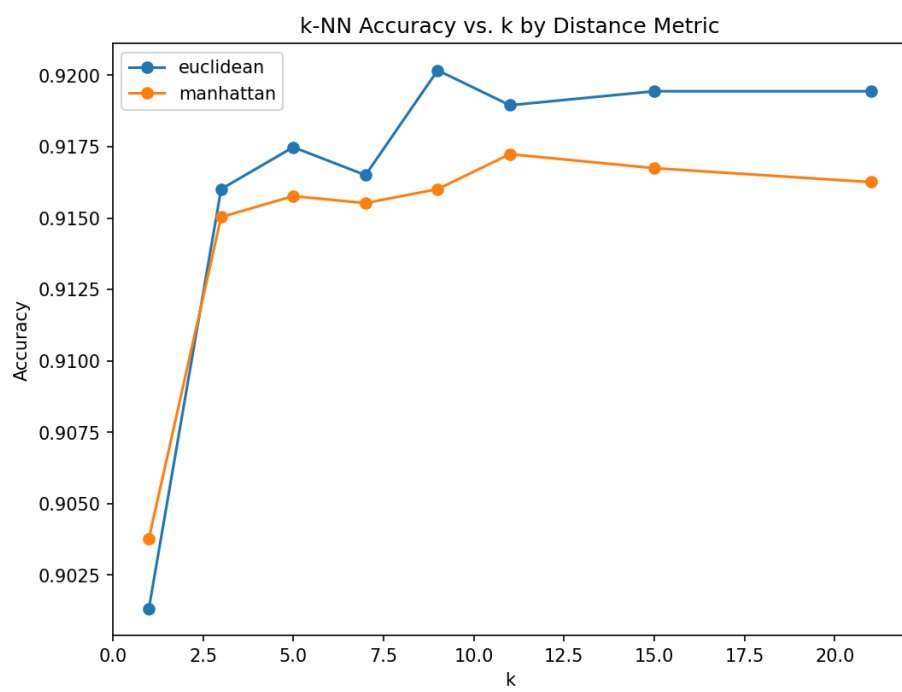


Figure 4: k-NN accuracy vs k

Figure 4:  $k$ -NN test accuracy as a function of  $k$ , for both distance metrics. Both curves rise sharply from  $k = 1$ , peak around  $k = 9$ – $15$ , and plateau thereafter, visually confirming the bias–variance trade-off discussed in Sections 4 and 7:  $k = 1$  slightly overfits to individual noisy training points, while accuracy stabilises once enough neighbours are averaged over to suppress that noise, and does not meaningfully degrade even out to  $k = 21$  because the classes that remain confusable (DERMASON/SIRA, BARBUNYA/CALI) are confusable at every neighbourhood scale tested.

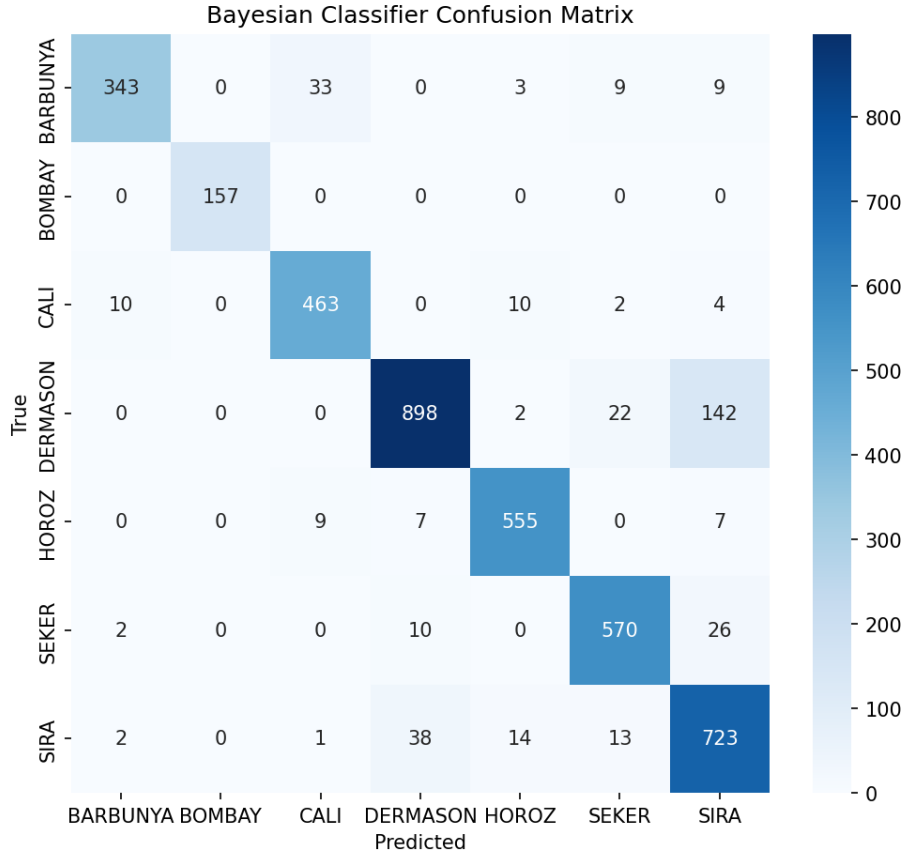


Figure 5: Bayesian confusion matrix

Figure 5: *Bayesian classifier confusion matrix.* The largest off-diagonal mass sits at (DERMASON, SIRA) = 142 and (SIRA, DERMASON) = 38, with a smaller concentration at (BARBUNYA, CALI) = 33. Every other class pair has single- or low-double-digit confusion counts, and BOMBAY’s row and column are entirely zero off the diagonal.

Figure 6: *Confusion matrix for the best  $k$ -NN configuration (Euclidean,  $k=9$ ).*

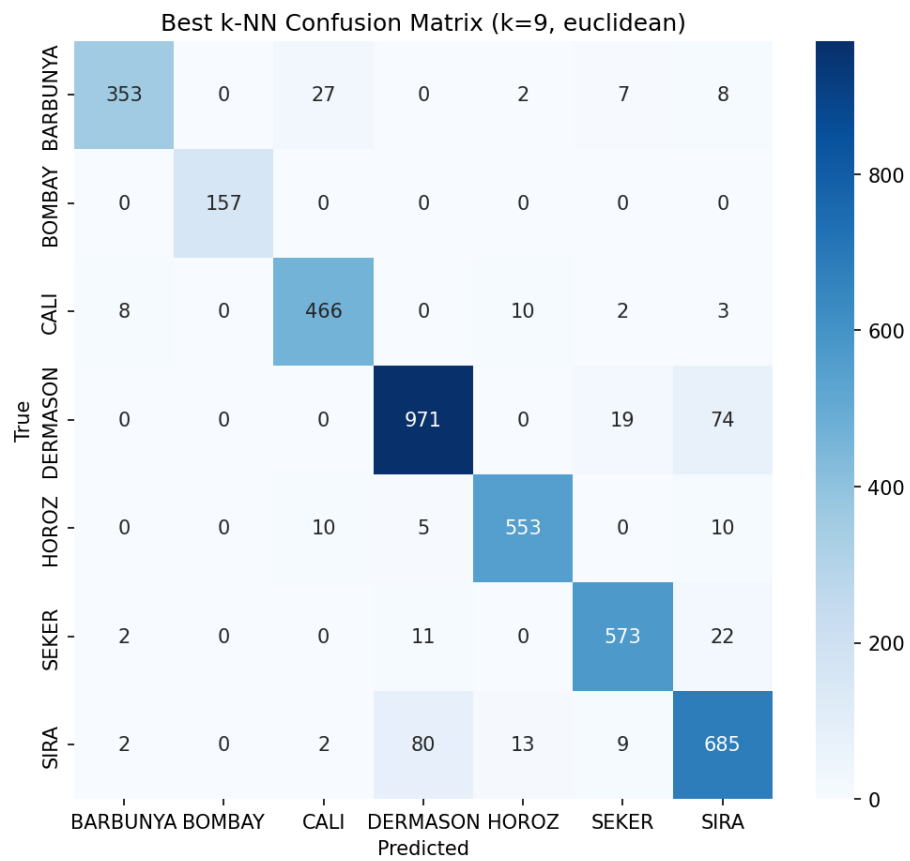


Figure 6: k-NN (best config) confusion matrix

The same two error clusters persist (DERMASON $\leftrightarrow$ SIRA, BARBUNYA $\leftrightarrow$ CALI) but are visibly smaller: DERMASON $\rightarrow$ SIRA drops from 142 (Bayesian) to 74, and BARBUNYA $\rightarrow$ CALI drops from 33 to 27. This is the main source of k-NN’s overall accuracy advantage: it does not eliminate the geometrically genuine overlap between these classes, but it resolves more of the borderline cases correctly than the Gaussian model does, because it makes decisions from local neighbourhood structure rather than from a single global ellipsoidal fit per class.

## 9. Critical Discussion of Limitations

**Gaussian assumption validity.** The Bayesian classifier’s core assumption, that each class’s 16 features jointly follow a multivariate Gaussian, is demonstrably violated for several of this dataset’s features. Univariate skewness computed on the raw features shows **Area** (skew  $\sim 2.95$ ), **ConvexArea** ( $\sim 2.94$ ), **MinorAxisLength** ( $\sim 2.24$ ) and **EquivDiameter** ( $\sim 1.95$ ) are strongly right-skewed, while **ShapeFactor4** ( $\sim -2.76$ ) and **Solidity** ( $\sim -2.55$ ) are strongly left-skewed; only a handful of features (**Compactness**, **ShapeFactor3**) are close to symmetric. A true multivariate Gaussian requires every univariate marginal to itself be Gaussian, so this skew is direct evidence the Gaussian model is a systematic approximation rather than a good fit, particularly for the size-based feature group. This matters because the Bayesian classifier’s quadratic discriminant boundary (Section 4.2) is derived entirely from each class’s estimated  $\mu_k, \Sigma_k$ ; if the true class-conditional density has heavier tails or skewed mass than a Gaussian, the fitted ellipsoidal decision regions will systematically mis-weight the tails, which is a plausible contributor to the Bayesian model’s larger DERMASON $\leftrightarrow$ SIRA confusion (Section 7/8) relative to k-NN.

**k-NN: curse of dimensionality, choice of k, and compute cost.** k-NN makes no distributional assumption but pays for that flexibility in two ways. First, in  $d = 16$  dimensions, distance concentration (the curse of dimensionality) means all pairwise distances tend to become more similar as  $d$  grows, which can weaken the discriminative power of “nearest” neighbours; the strong pairwise feature correlations documented in Section 2 partially mitigate this here because the *effective* dimensionality of the data is lower than 16 (several features carry near-duplicate information), but this also means k-NN is implicitly relying more heavily on a smaller number of independent directions than the raw feature count suggests. Second,  $k$  is a hyperparameter that must be tuned (Section 7 shows accuracy is not monotonic in  $k$  and is sensitive at the low end), and no separate validation set was held out purely for tuning  $k$  in this study: the reported “best”  $k = 9$  is selected by test-set accuracy directly, which is standard for a course project sweep but would in a production setting risk a mildly optimistic estimate of generalisation accuracy for that specific  $k$ . Third, k-NN is a **lazy learner**: prediction requires computing the distance from each query point to all 9,527 training points ( $O(N)$  per query, and  $O(N \cdot d)$  accounting for the 16 feature dimensions), whereas the Bayesian classifier’s prediction cost is

$O(1)$  per query per class (7 Mahalanobis-distance evaluations against precomputed inverse covariance matrices) after a one-time  $O(Nd^2 + d^3)$  fitting cost to estimate and invert the per-class covariances. For 4,084 test queries this difference is immaterial in absolute wall-clock time, but it would not scale gracefully to a much larger deployment dataset without approximate nearest-neighbour indexing.

**Class imbalance.** BOMBAY (522 samples, 3.8% of the dataset) is nearly  $7\times$  smaller than DERMASON (3,546 samples, 26.1%). In this particular dataset the imbalance turned out not to hurt BOMBAY’s classification: it is perfectly classified in every configuration because it is geometrically an outlier class (Section 8), but this is a property of this specific dataset’s feature geometry, not a general guarantee. Macro-averaged precision/recall/F1 (used throughout Section 7) were deliberately chosen over micro-averaging or plain accuracy specifically because they weight every class equally regardless of its sample count, which is the correct choice under this imbalance; had a smaller, less geometrically separable minority class been present, the same imbalance could easily have produced a large gap between accuracy (which BARBUNYA/CALI/DERMASON/SIRA dominate numerically) and macro F1 (which would penalise poor performance on the minority class equally). The stratified train/test split (Section 2/6) ensures BOMBAY is at least proportionally represented in both splits, which mitigates but does not eliminate the general risk that a class with few training examples yields noisier mean/covariance estimates (Bayesian) or a thinner local neighbourhood (k-NN).

## 10. Central Research Question

**How do assumptions about the underlying probability distribution affect the performance of parametric and non-parametric pattern classifiers?**

The empirical result in this study is that the non-parametric k-NN classifier outperformed the parametric Gaussian Bayesian classifier at 14 of the 16 tested ( $k$ , metric) configurations on both accuracy and macro F1: every configuration except  $k = 1$  under either distance metric, where the single-neighbour rule is too noisy to beat the smoothed Bayesian estimate. For any  $k > 1$  in the sweep, k-NN wins outright, with the best configuration (Euclidean,  $k = 9$ ) beating the Bayesian classifier by 1.2 accuracy points (92.02% vs 90.82%, 3,758 vs 3,709 correct out of 4,084). In other words, the finding is not that non-parametric classification is unconditionally superior regardless of hyperparameter choice, but that once k-NN is given enough neighbours to smooth out single-point noise, its lack of a distributional assumption becomes a decisive advantage on this dataset.

This gap is directly explainable by the distributional mismatch documented in Section 9. The Bayesian classifier’s entire decision function (Section 4.2) is derived from the assumption that each class’s 16 features are jointly Gaussian;

but several of the Dry Bean dataset’s size-based features (**Area**, **ConvexArea**, **MinorAxisLength**, **EquivDiameter**) show marked skew ( $|\text{skew}| > 1.9$ ), and the strong near-collinearity among size features (Area–ConvexArea  $r = 0.9999$ ) and among shape features (Compactness–ShapeFactor3  $r = 0.9987$ ) makes the per-class covariance matrices used by the Bayesian model ill-conditioned in several directions (condition numbers of roughly  $1 \times 10^7$ – $1.3 \times 10^8$  per class). The `reg_covar = 1e-6` ridge is applied as a defensive numerical-stability measure rather than out of strict necessity: on this dataset the covariance matrices remain positive-definite and invertible even with `reg_covar=0`, and removing the ridge changes test accuracy by only 0.02 percentage points (90.84% vs 90.82%). When the true class-conditional density departs from the assumed Gaussian shape, as it demonstrably does here, the fitted ellipsoidal decision boundaries are a systematically imperfect approximation of the true (unknown) class boundaries, and this shows up concretely as the Bayesian model’s larger DERMASON<->SIRA confusion (142 misclassifications vs 74 for the best k-NN model, Section 7/8): these two classes overlap in exactly the collinear, non-Gaussian size features that the Bayesian model must approximate with an ellipsoid.

k-NN, by contrast, makes no parametric assumption at all: it estimates the local class density non-parametrically from whichever training points happen to be nearby in standardised Euclidean space, so it can trace decision boundaries of arbitrary shape, including boundaries that follow the actual skewed, non-elliptical spread of the DERMASON/SIRA/BARBUNYA/CALI feature clouds visible in the PCA scatter (Figure 3), at the cost of requiring more computation per query and being sensitive to the choice of  $k$  and to the curse of dimensionality (Section 9). In this dataset, where feature distributions are demonstrably non-Gaussian but the effective dimensionality is reduced by strong feature redundancy, that trade-off favours the non-parametric model: the flexibility to model non-elliptical class regions outweighs the variance cost of a purely local decision rule. Where the underlying class-conditional distributions are close to Gaussian (or where training data is too scarce to support a reliable local neighbourhood estimate), the balance would be expected to tip back toward the parametric model, consistent with the well-known result that parametric models dominate when their distributional assumption holds and lose their advantage (or become actively misleading) when it does not.

## References

- Koklu, M. and Ozkan, I.A. (2020). “Multiclass Classification of Dry Beans Using Computer Vision and Machine Learning Techniques.” *Computers and Electronics in Agriculture*, 174, 105507.
- Dua, D. and Graff, C. (2019). UCI Machine Learning Repository, Dry Bean Dataset. Irvine, CA: University of California, School of Information and Computer Science. <https://archive.ics.uci.edu/dataset/602/dry+bean+dataset>
- Duda, R.O., Hart, P.E., and Stork, D.G. (2001). *Pattern Classification*

- (2nd ed.). Wiley. (Bayes decision theory, Gaussian discriminant analysis, and k-NN, as taught in DSCD612.)
- Pedregosa, F. et al. (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, 2825-2830. (Used for train/test splitting, standard scaling, label encoding, and evaluation metrics only, not for the classifiers themselves; see Section 5.)